

CSC 648/848 Software Engineering Spring 2026

Pathwise

Section 2

Team 2

Omeid Nadery — Team Lead — onadery@sfsu.edu

Jason Le — Back End Lead — jle34@sfsu.edu

Damian Perez — Front End Lead — dperez52@sfsu.edu

Brandon Sanchez — Bsanchez15@sfsu.edu

Laura Wong — lwong25@sfsu.edu

Mikias Berhane — mberhane1@sfsu.edu

Cielina Maree Lubrino — clubrino@sfsu.edu

Milestone 2

3/10/26

History Table (Revision)

Name	Revision Summary	Revision Date
Laura	Started on <i>Executive Summary</i>	2/10/2026
Omeid	Completed <i>Initial List of Functional Requirements</i> and copied <i>Non-Functional Requirements</i>	2/10/2026
Damian	Started on <i>Main Data Items and Entities</i>	2/12/2026
Cielina	Started on <i>Personas and Use Cases</i>	2/12/2026
Brandon	Started <i>High-level System Architecture and Technologies Used</i>	2/13/2026
Jason Le	Started <i>Competitive analysis</i>	2/14/2026
Damian	Finished <i>Main Data Items and Entities</i> ; updated document formatting	2/17/2026
Laura	Finished <i>Executive Summary</i>	2/17/2026
Brandon	Finished <i>High-level System Architecture and Technologies Used</i>	2/17/2026
Mikias	Revised <i>Competitive Analysis</i>	2/17/2026
Omeid	Modified functional requirements and grouped them in priorities	3/1/2026
Cielina	Added <i>UML Diagrams</i>	3/2/2026
Damian	Added completed wireframes and storyboard to <i>UI Mockups and Storyboard</i>	3/3/2026
Cielina	Added completed wireframes and storyboard to <i>UI</i>	3/3/2026

	<i>Mockups and Storyboard</i>	
Brandon	Added completed wireframes and storyboard to <i>UI Mockups and Storyboard</i>	3/3/2026
Damian	Added further business value justification to <i>Executive Summary</i>	3/6/2026
Mikias	Finished the <i>Risks & Project Management</i>	3/6/2026
Omeid	Added Appendix I and finished High Level Architecture, DB Organization	3/7/2026
Damian	Added Frontend Team Support to <i>Teams and Roles</i>	3/9/2026

Functional Requirements - Prioritized

PRIORITY 1 (Must Have)

Actor: Registered Students (Undergraduate, Graduate, International)

FR_01

The system shall allow users to create, authenticate, and manage a personal account using secure session-based login.

FR_02

The system shall allow registered users to create, edit, delete, and categorize goals with title, description, category, target completion date, and status.

FR_02.1

The system shall allow users to change goal status between active, paused, and completed with a listed selection button click.

FR_03

The system shall allow users to create projects associated with a specific goal.

FR_03.1

Each Project shall contain one or more milestones with title, description, target date, and completion status.

FR_04

The system shall allow users to attach resources, templates, or workflows to a specific goal or project.

FR_05

The system shall provide a searchable discovery engine that allows users to search resources, templates, and workflows using keyword input.

FR_06

The system shall allow users to filter search results by predefined attributes including category, content type, skill area, and AI-enabled status.

FR_07

The system shall display search results with relevant metadata including tags, ratings, and descriptions to support informed resource selection.

FR_08

The system shall allow users to bookmark and save resources for later access.

FR_09

The system shall provide a personalized dashboard that displays a user's active goals, associated projects, upcoming milestones, and saved resources in a single interface after login.

FR_10

The system shall display the completion status of goals and milestones using visual indicators (e.g., progress bars or status labels) to show the percentage or state of completion.

Actor: Guest User

FR_21

The system shall allow guest users to browse publicly available categorized content without registration

PRIORITY 2 (Desired)

Actor: Registered Students

FR_11

The system shall allow users to create and store reflections associated with goals or projects.

FR_12

The system shall record user activities, including goal creation, milestone completion, resource bookmarking, and reflections, and provide a chronological summary of these activities over a selected time period.

FR_13

The system shall allow registered users to create and update a personal profile containing information such as academic interests, career goals, and areas where assistance is needed.

FR_14

The system shall generate recommendations for resources, workflows, and goal templates based on a user's stored profile information, interests, and recorded activity history.

FR_18

The system shall allow users to rate and assign descriptive tags to shared resources, workflows, and templates to improve discoverability within search results.

FR_20

The system shall generate summary reports that aggregate user activity data, including completed goals, milestone progress, and reflections, over defined time periods such as weekly, monthly, or semester views.

FR_17

The system shall provide in-site messaging functionality to enable communication between Registered Users.

Actor: Faculty / University Support Staff

FR_19

The system shall allow Faculty and Support Staff users to publish categorized resources and templates for student discovery.

PRIORITY 3 (Opportunistic)

Actor: Registered Students

FR_15

The system shall allow users to browse public community-shared goal templates and workflows.

FR_16

The system shall allow registered users to share resources, workflows, or goal templates with other registered users within the platform.

FR_22

The system shall display a visible indicator identifying whether a listed resource or tool uses artificial intelligence.

UI Mockups and Storyboards

UC_1: Dashboard & Goal Creation

Storyboard by Cielina Lubrino

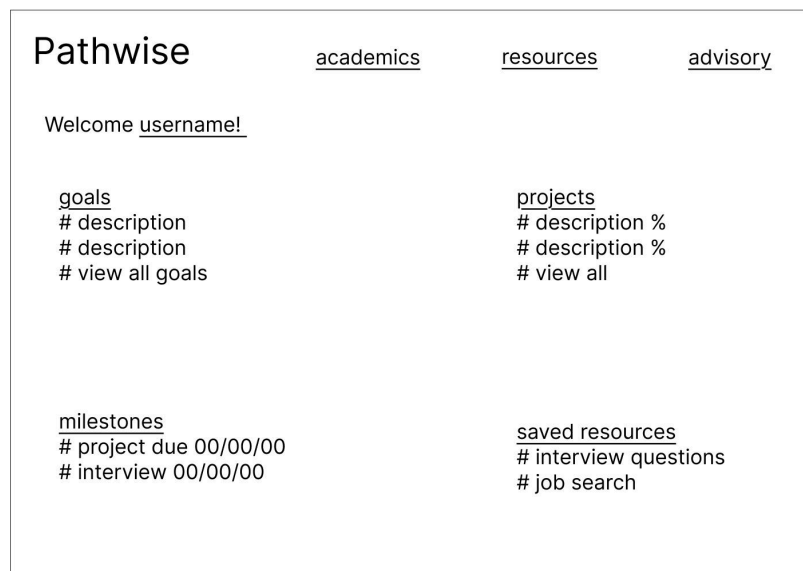
Use Case Summary

A registered student logs into Pathwise and views a personalized dashboard summarizing their active goals, projects, milestones, and saved resources. The dashboard provides a high-level progress overview to support structured planning and reduce cognitive overload.

Screen 1 — Main Dashboard Overview

The Dashboard loads with a top navigation bar containing Academics, Resources, and Advisory. The main content area displays a welcome message with the user's username followed by four structured sections: Goals, Projects, Milestones, and Saved Resources. Each section presents a summarized preview with a "View All" link to navigate to detailed pages.

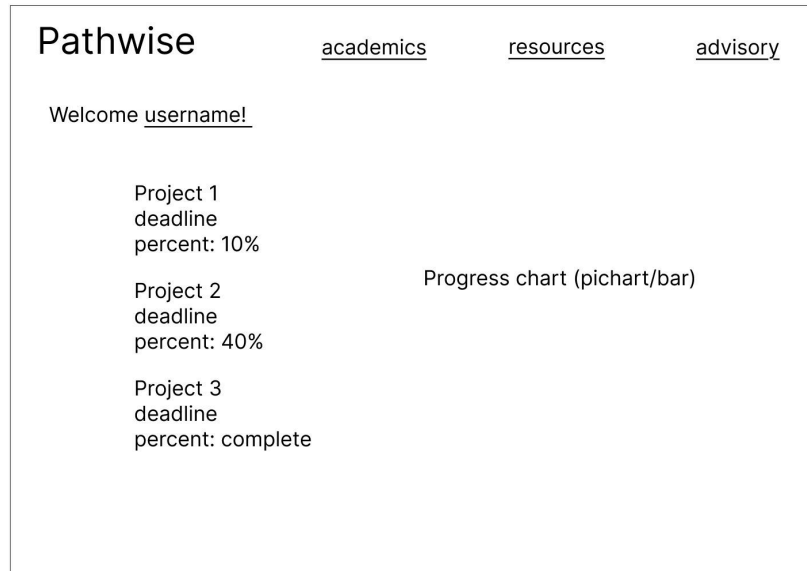
Requirements addressed: FR_09, FR_10



Screen 2 — Projects & Progress Overview

The Projects section displays individual project cards containing the project title, deadline, and percentage completion. A visual progress representation (bar or pie placeholder) appears alongside the project list to provide an at-a-glance summary of overall completion. Users may select a project to view associated milestones.

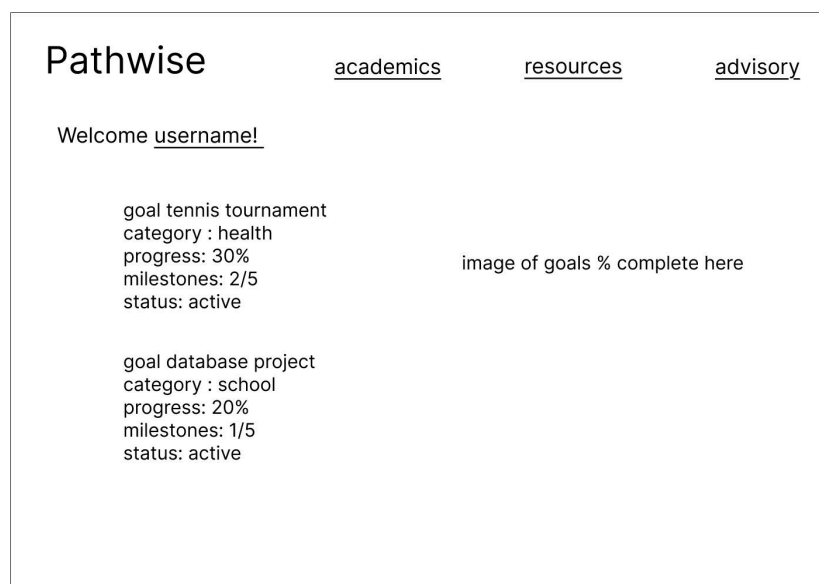
Requirements addressed: FR_03, FR_03.1, Fr_10



Screen 3 — Goal List with Progress Details

The Goals page displays individual goal entries including title, category, progress percentage, milestone completion count, and status. A visual placeholder for overall goal completion appears alongside the goal list. This layout enables structured tracking of goal advancement and status updates.

Requirements addressed: FR_02, FR_02.1, FR_10



Screen 4 — User Profile Overview

The Profile page displays the user's name, student ID/employee ID, department or major, and optional visa status alongside a profile image placeholder. The lower section includes important notifications, new messages, saved resources, and a bulletin board area for announcements. This screen centralizes personal and communication-related information.

Requirements addressed: FR_01, FR_13, FR_17

Pathwise		academics	resources	advisory
Welcome <u>username!</u>				
Image	name: student id/employee id major/dept (if app visa status)			
Important! New Messages Saved Resources	Bulletin Board			

UC_2: Search & Resource Discovery

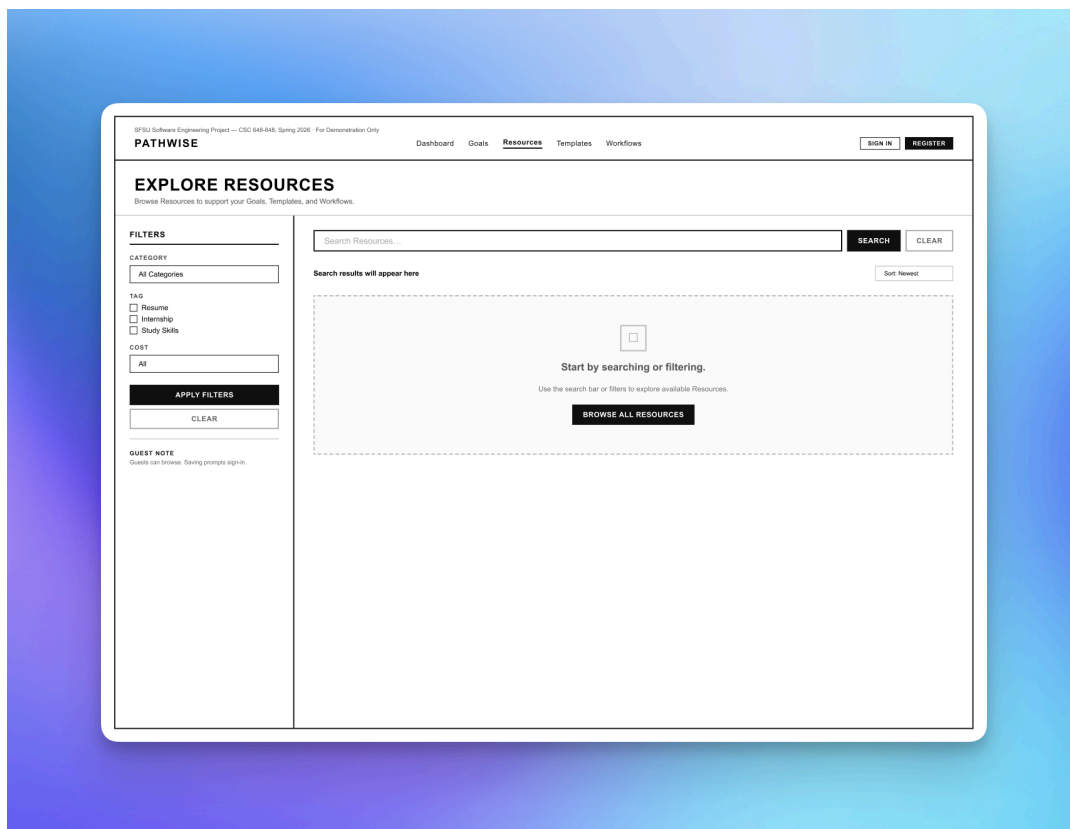
Storyboard by Damian Perez

Use Case Summary: A student uses the Pathwise discovery engine to search for Resources by keyword, apply filters, and evaluate results aligned with their current Goals. Registered users can bookmark Resources directly. Guest users can browse and filter but must sign in to save.

Screen 1 — Resource Discovery (Browse & Search)

The Resources page loads with a Filter Sidebar on the left and a Content Area on the right. The sidebar contains five filter groups: Category, Tag, Content Type, AI-Enabled, and Cost Type. The Content Area contains a keyword search bar and displays an empty browse state by default. Clicking Browse All Resources returns the full unfiltered Resource list.

Requirements addressed: FR_05, FR_06, FR_21, FR_22.

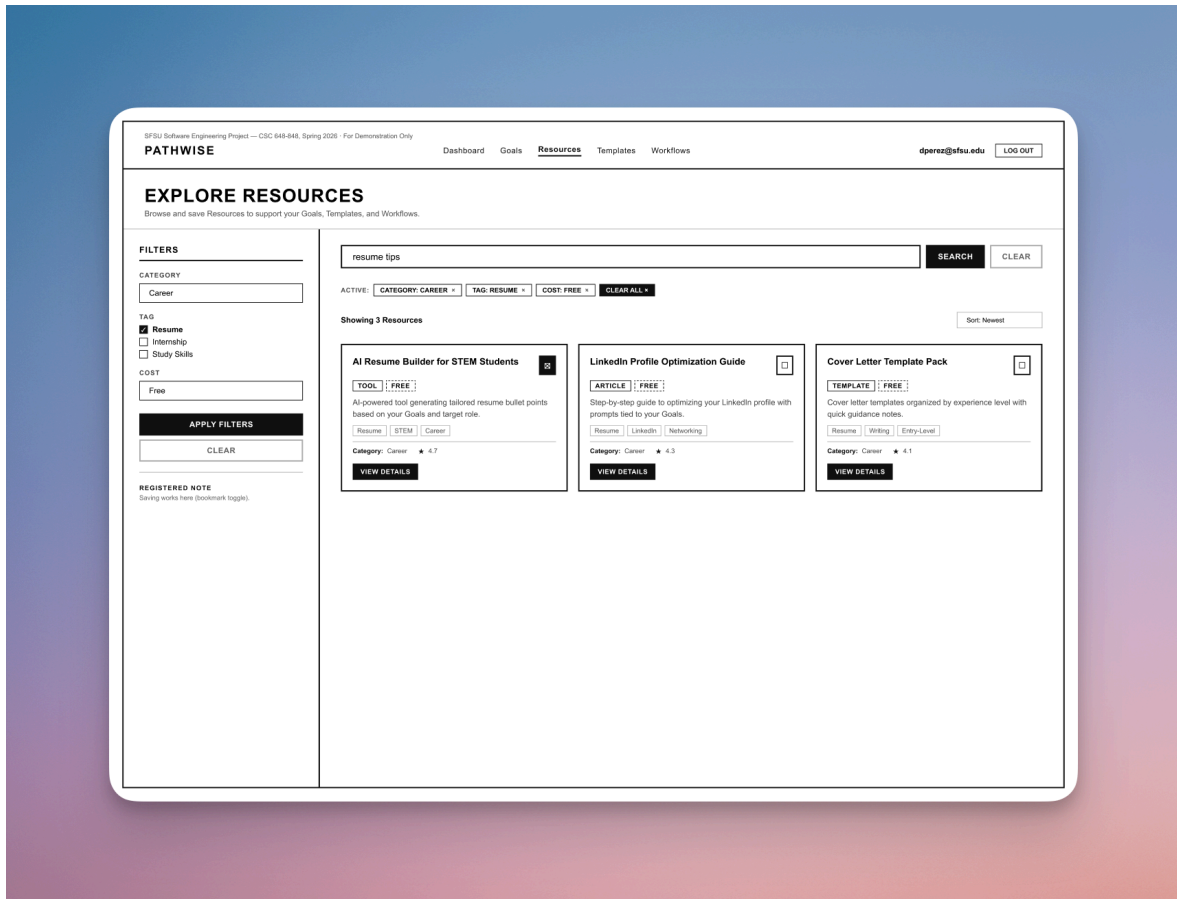


Screen 2 — Filtered Results State

After a search or filter is applied, active filters appear as removable chips above a three-column Resource Card grid. Each card shows the Resource title, description, badges for AI-Enabled status, Content Type and Cost Type, Tags, and a rating. Registered users can bookmark from the card. Guest users see a

disabled bookmark that triggers a sign-in prompt. Zero results shows an empty state. A fetch failure shows a non-blocking error alert with a Retry control.

Requirements addressed: FR_05, FR_06, FR_08, FR_21, FR_22.



Screen 3 — Resource Detail and Save

Clicking View Details opens the full Resource record. The left column shows the complete description, linked Goal and Template counts, an outbound Access Resource link, and a Related Tags row. The right column shows a Save Panel — a bookmark toggle for registered users or a sign-in prompt for guests — and a Metadata Panel listing Content Type, Cost, Category, and AI-Enabled status.

Requirements addressed: FR_05, FR_08, FR_21, FR_22.

Resources > AI Resume Builder for STEM Students
[← Back to Resources](#)

AI Resume Builder for STEM Students

TOOL FREE CATEGORY: CAREER

★★★★★ 4.7/5 (43 ratings)

ABOUT THIS RESOURCE

Generates tailored resume bullet points based on your skills, Goals, and target role. Output is editable and exportable.

128

GOALS LINKED

14

TEMPLATES USING THIS

ACCESS

[ACCESS RESOURCE](#)

Opens in a new tab.

RELATED TAGS

[Resume](#) [STEM](#) [Career](#) [Internship](#) [Entry-Level](#)

SAVE THIS RESOURCE

[SAVE TO MY RESOURCES](#)

Saved Resources appear in your Dashboard and can be linked to Goals and Templates.

Guest users see a sign-in prompt instead of saving.

CONTENT TYPE	Tool
COST	Free
CATEGORY	Career

UC_3: Reviewing Progress and Reflecting on Performance

SFSU Software Engineering Project — CSC 648-848, Spring 2026 · For Demonstration Only
PATHWISE
Dashboard Goals **Resources** Templates Workflows LOG OUT

PATHWISE - Student Dashboard

Complete CIS 648
Due: March 2, 2026 | Status Active
Overall Progress

75% complete
Milestones (Final Project)
☒ Complete Milestone 1
☐ Complete Milestone 2

Complete CIS 665
Due: March 5, 2026 | Status Active
Overall Progress

50% complete
Milestones (Final Project)
☒ Review chapter 1-5

RECENT ACTIVITY
Actions this week
Feb 28 - Complete milestone 1
March 2 - Complete milestone 2
March 5 - Review chapter 1-5

+ Add Reflection

Screen 1- Progress

The purpose of this page is for the student to review overall progress on all their goals and milestones in one place. Student can log into dashboard -> review active goals and progress -> see completed and pending milestones, they can also check and see the recent activity and be able to add a reflection
Requirements addressed: FR_09, FR_10, FR_12

SFSU Software Engineering Project — CSC 648-848, Spring 2026 - For Demonstration Only

PATHWISE

DashboardGoalsResourcesTemplatesWorkflows

LOG OUT

PATHWISE - Add Reflection

Journal Your Progress

Write your reflection *

Share your thoughts about your progress, challenges, and next steps. This will be saved to your personal journal.

I've made good progress on chapter 1-5. The Practice problem helped clarify the mechanisms

Associate with Goal or Project(Optional)

Link this reflection to a specific goal or project for better organization.

--Select a goal or project --

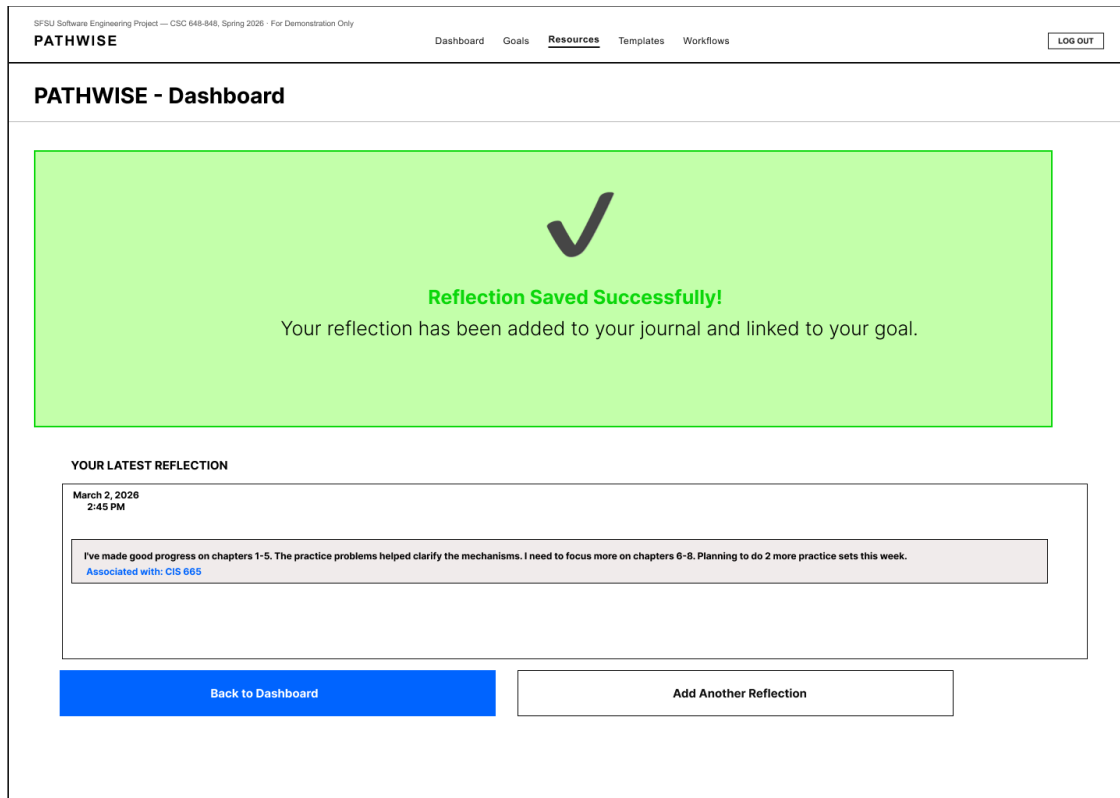
Submit Reflection

Cancel

Screen 2- reflection page

The purpose for this page is for students to be able to write a journal reflection about their progress, challenges and future plans. After viewing their progress page, students can hit add reflection which then takes them to this screen where the form opens up. Student also has the option to add this reflection to a specific goal or project

Requirements addressed: FR_11, FR_04



Screen 3- Confirmation

The purpose of this page is just to show that the student reflection was uploaded successfully and saved. After submitting the form. System will save onto database and students will see a confirmation and have the opportunity to add another or return to dashboard

Requirements addressed: FR_11, FR_12

Storyboard by Brandon Sanchez

UC_4: Supporting Advising and Academic Guidance

SFSU Software Engineering Project — CSC 648-848, Spring 2026 - For Demonstration Only

PATHWISE

DashboardGoalsResourcesTemplatesWorkflows

LOG OUT

PATHWISE - Advising Session

Student: Brandon Sanchez| Advisor: i dont know| Date: March 2, 2026

Complete CIS 648

Due:March 2, 2026 |Status Active

Overall Progress

75% complete

Milestones (Final Project)

☒ Complete Milestone 1

☐ Complete Milestone 2

Prepare for Internship

Due:May 31, 2026 |Status Active

Overall Progress

50% complete

Milestones (Final Project)

☒ Research company

☐ Update resume

RECENT ACTIVITY

Students Progress this week

Feb 28 - Complete milestone 1

March 2 - Complete milestone 2

March 5 - Review resume

Session Notes

Document key discussion points, recommendations , and next steps

+ Add comment

Storyboard by Brandon Sanchez

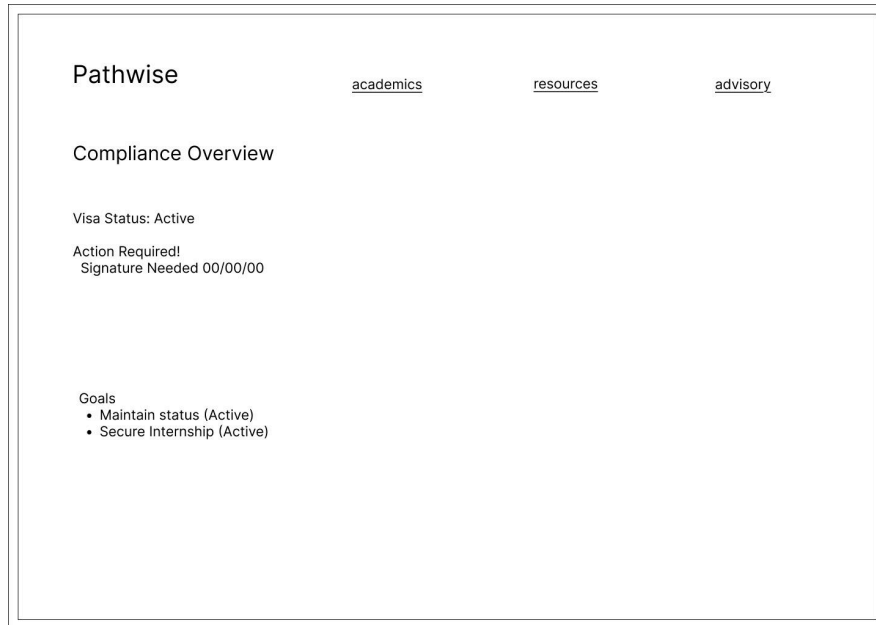
Screen 1- advising view

Serves as a shared reference point during advising sessions where faculty and students can discuss future plans and academic planner. Faculty can also leave comments for students to view.

Requirements addressed: FR_09, FR_10, FR_12, FR_04

UC_5: Managing Academic and Compliance Responsibilities for International Students

Storyboard by Cielina Lubrino



Screen 1— Compliance Overview

The Compliance Overview page displays the student's current visa status and any required actions. Below the status summary, active goals related to immigration and career planning are listed to reinforce the connection between compliance and long-term academic success.

Requirements addressed: FR_02, FR_02.1, FR_09, FR_10

High level Architecture, Database Organization

The Pathwise application follows a relational database design implemented using MySQL on AWS Relational Database Service. The schema reflects our core data entities, terminology including Goals, Projects, Milestones, Resources, Templates, Workflows, Reflections, Messages, ActivityLogs, Categories, and Tags, and supports our data relations for example goals containing projects, projects containing milestones, categories and tags for resources, and etc defined in milestone 1.

Core User Tables)

PK = Primary Key, FK = Foreign Key (Important for table relationships)

- Users:

Stores all registered users, including students, faculty, and support staff.

Main fields: userId (PK), name, email, passwordHash, role (student, facultyStaff), createdAt, accountStatus.

- UserProfiles:

Stores personalization data associated with a user.

Main fields: profileId (PK), userId (FK), interests, challenges, workloadLevel, aspirations, updatedAt.

Goal Planning Tables)

- Goals:

Stores user-defined objectives.

Main fields: goalId (PK), userId (FK), title, description, categoryId (FK), targetDate, status (active, paused, completed), createdAt.

- Projects:

Structured plans associated with a specific Goal.

Main fields: projectId (PK), goalId (FK), userId (FK), name, description, startDate, endDate, status.

- Milestones:

Checkpoints within a Project.

Main fields: milestoneId (PK), projectId (FK), title, description, targetDate, status, completedAt.

Discovery and Content Tables)

- Resources:

External or internal content supporting academic or professional goals.

Main fields: resourceId (PK), title, description, url, imagePath, categoryId (FK), isAiEnabled (Boolean), visibility (public/private), createdByUserId (FK), createdAt.

- Templates:

Pre-built goal or project frameworks.

Main fields: templateId (PK), title, description, categoryId (FK), visibility (public/private), createdByUserId (FK), createdAt.

- Workflows:

Step-by-step guides for completing structured processes.

Main fields: workflowId (PK), title, description, stepsText, categoryId (FK), visibility (public/private), createdByUserId (FK), createdAt.

Organization and Classification Tables)

- Categories:

Broad content classifications such as academic, career, health, personal development.

Main fields: categoryId (PK), categoryName.

- Tags:

Specific descriptors used for filtering and search refinement.

Main fields: tagId (PK), tagName.

- ItemTags:

Join table linking Tags to Resources, Templates, Workflows, or Goals.

Main fields: itemTagId (PK), itemType, itemId, tagId (FK).

User Interaction Tables)

- SavedResources:

Stores bookmarked resources for a user.

Main fields: savedResourceId (PK), userId (FK), resourceId (FK), savedAt.

- Reflections:

Journal-style entries created by users.

Main fields: reflectionId (PK), userId (FK), goalId (FK, nullable), projectId (FK, nullable), contentText, createdAt.

- **Messages:**

In-site communication between registered users.

Main fields: messageId (PK), senderUserId (FK), recipientUserId (FK), contentText, sentAt, readStatus.

- **ActivityLogs:**

System-generated records of user actions.

Main fields: logId (PK), userId (FK), actionType, entityType, entityId, createdAt.

Relationship Summary)

- A User owns multiple Goals.
- Goals contain one or more Projects.
- Projects contain multiple Milestones.
- Resources, Templates, and Workflows may be linked to Goals or Projects.
- Reflections optionally reference a Goal or Project.
- Messages are exchanged between registered Users.
- ActivityLogs aggregate user actions across the system.

This schema supports personalization, search functionality, content discovery, advising workflows, and dashboard analytics while maintaining a clean relational structure.

Media Storage)

Pathwise will store images in the server file system while maintaining image reference information within the MySQL database. The database will not store image BLOBs. Instead, each content item that supports media, such as a Resource, will include an imagePath field that stores the file system location or URL reference of the associated image. For example, a Resource entry may include an imagePath value such as /uploads/resources/resume101.png. The actual image file will be stored in the server directory structure, while the database stores only the reference path. When search results or dashboards are rendered, the backend will retrieve the imagePath value and send it to the frontend, which will then request the image directly from the web server.

This approach keeps the database lightweight, simplifies backups, improves performance for media delivery, and aligns with scalable web application practices. It also allows images to be optional. If a Resource does not include an imagePath, the system will display a default placeholder image. Video/Audio/GPS to be determined, for now not implemented.

Search/Filter Architecture and Implementation)

Pathwise will implement search and filtering using MySQL queries executed by the Express.js backend. The current version of search will use SQL keyword matching with %LIKE, combined with simple filters such as Category and Content Type. This approach is sufficient for the Vertical Prototype and establishes the end-to-end infrastructure needed for later expansion.

Search Scope and Searchable Items)

The search feature will focus on discovery content that supports student development and planning. Searchable items include:

- **Resources**
- **Templates**
- **Workflows**

These items will be returned as unified results labeled by type (Resource, Template, Workflow), or optionally grouped by type for readability.

Search Fields and DB Terms)

Search results will be computed by matching user keywords against key text fields stored in the database. At a minimum, the system will search:

- **title**
- **description**

Additionally, search will support Tags by joining against the Tags and ItemTags tables. This enables users to locate items using keywords that may not appear directly in the title.

Filters and Result Organization)

The user-facing search interface will support filtering using a dropdown selection and optional additional filters. The initial filter set will include:

- **Category** (Academic, Career, Wellness, Personal Development)
- **Content Type** (Resource, Template, Workflow)
- **AI-enabled flag** (applies to Resources only, when relevant)

Search results will display basic metadata including title, short description, category, tags (when available), and an image thumbnail or placeholder. This ensures results are informative without requiring detailed navigation.

Query Execution and Behavior)

When a user submits a search, the backend will construct SQL queries that apply:

- keyword matching using %LIKE on title and description
- optional category filtering using equality checks
- optional tag filtering using JOIN operations
- optional content type selection to query one or multiple entity tables

If the user submits an empty search term and no filters, the system will return a default list of all public discovery items.

APIs)

Pathwise will implement application-specific APIs in the Express.js backend to support core functionality. These APIs provide clear interfaces between the frontend and the MySQL database for authentication, planning workflows, discovery search, saving resources, and communication. The following endpoints represent the primary APIs Pathwise will implement at a high level:

POST /api/auth/register

The system shall allow a new user to create an account by submitting registration information, resulting in a new user profile being created.

POST /api/auth/login

The system shall authenticate a user using submitted credentials and establish a session-based login state.

POST /api/auth/logout

The system shall end the user session and log the user out of the application.

GET /api/dashboard

The system shall return the data needed to render the user's dashboard, including active goals, projects, milestones, and saved resources.

POST /api/goals

The system shall allow a logged-in user to create a new Goal with essential attributes such as title, description, category, and target date.

POST /api/projects

The system shall allow a logged-in user to create a Project associated with an existing Goal, supporting milestone-based planning.

GET /api/search

The system shall allow users to search discovery content using a keyword and optional filters such as category and content type. Results shall include basic metadata such as title, description, category, tags, and image reference information.

POST /api/saved-resources

The system shall allow a logged-in user to save or bookmark a Resource for later access, enabling the saved resources view on the dashboard.

These APIs represent the primary backend interfaces for the Vertical Prototype and for future milestones. Additional APIs for reflections, messaging, content publishing, tagging, and ratings may be introduced as the system expands.

Significant Non-Trivial Algorithms and Processes)

These are our current structured processes that support discovery and user progress beyond simple data storage and are relevant to our vertical prototype. These processes are subject to change in future milestones, possibly adding new ones and/or modifying current ones.

Search Relevance Ranking

Search results will be ranked using basic relevance logic. Matches found in an item's title will be treated as more significant than matches found only in the description. Filtering by category or tag will further refine results.

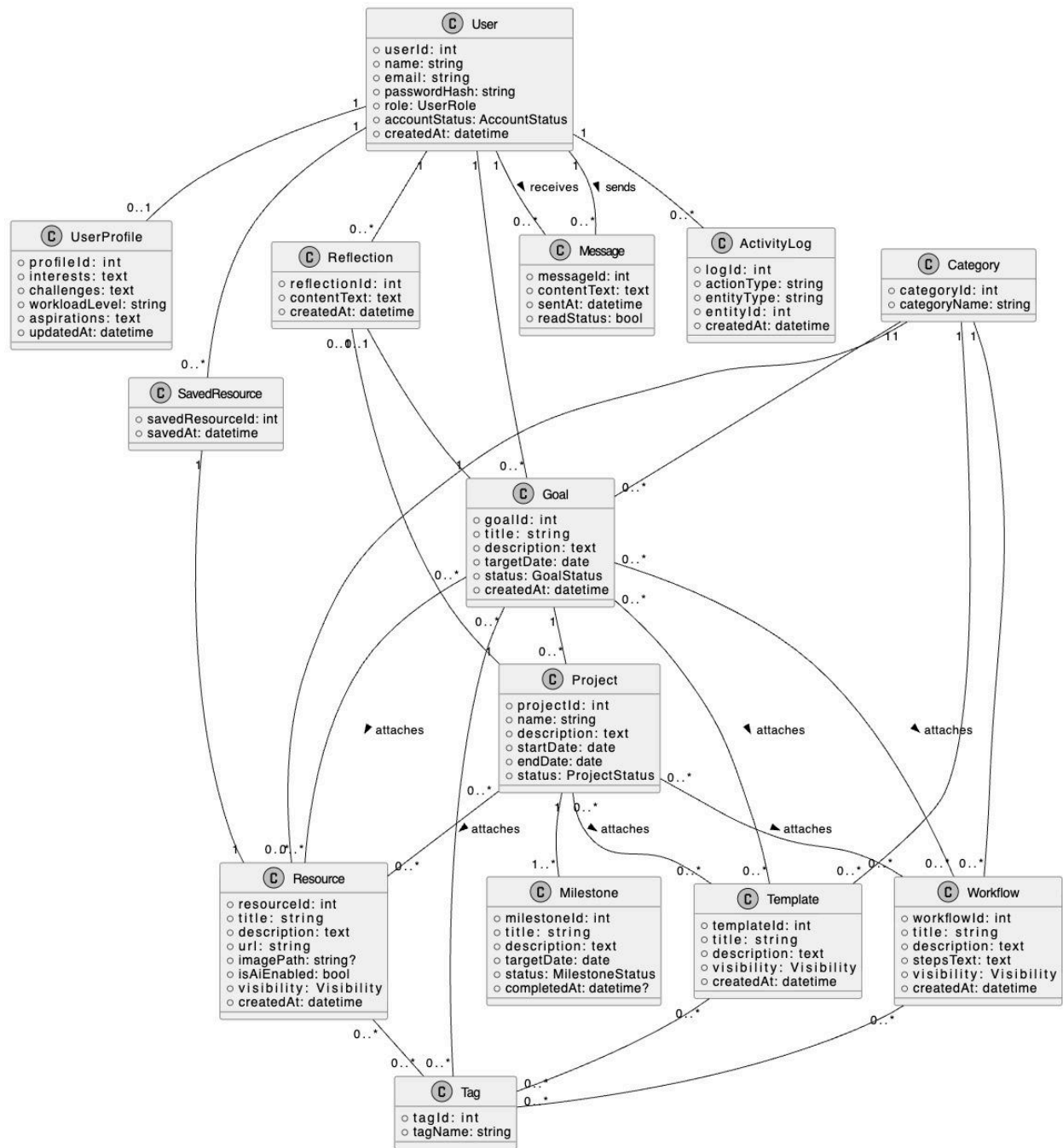
Rating Aggregation

If ratings are enabled, the system will calculate and display the average rating for each Resource or Template based on user submissions.

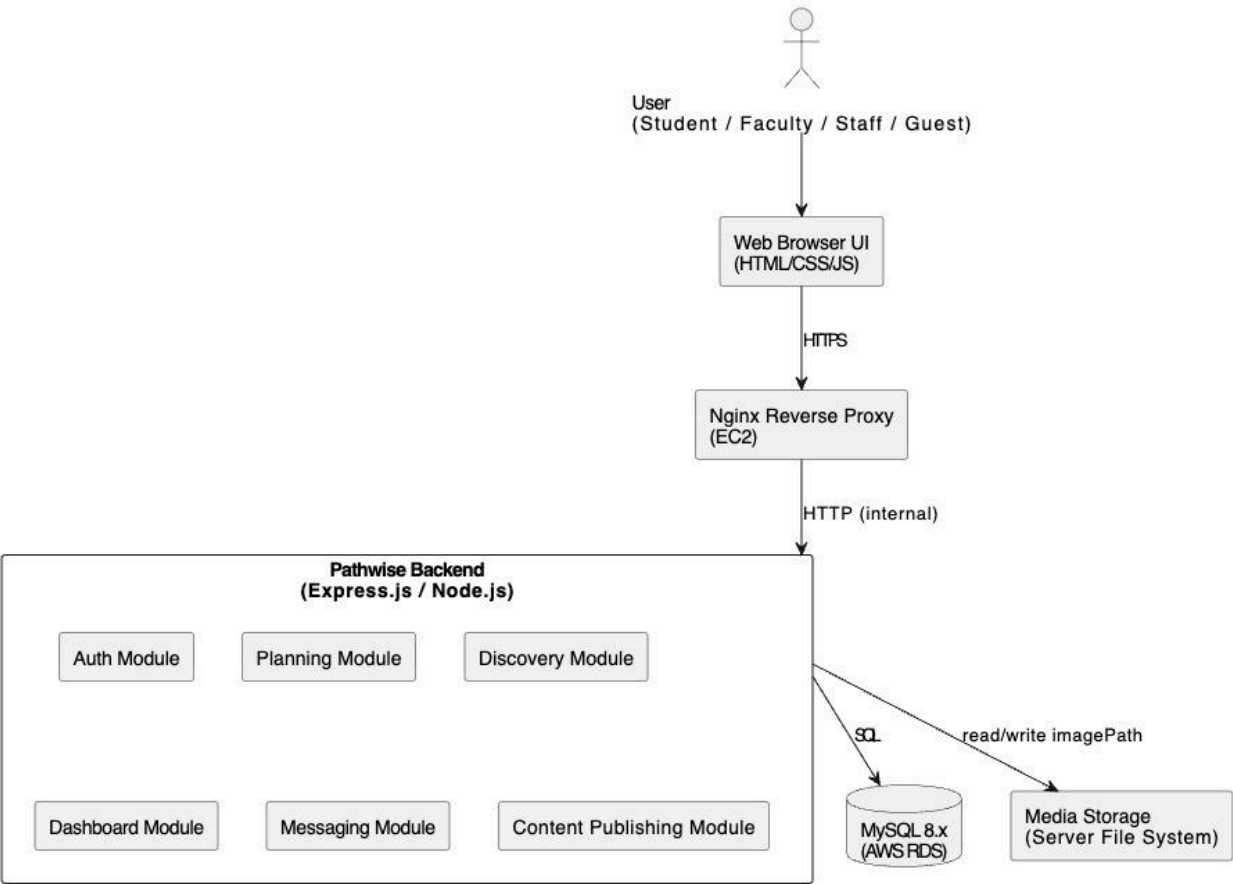
Activity-Based Progress Summary

User dashboards will generate progress summaries by aggregating milestone completion and ActivityLog data. This provides a structured overview of goal development over time.

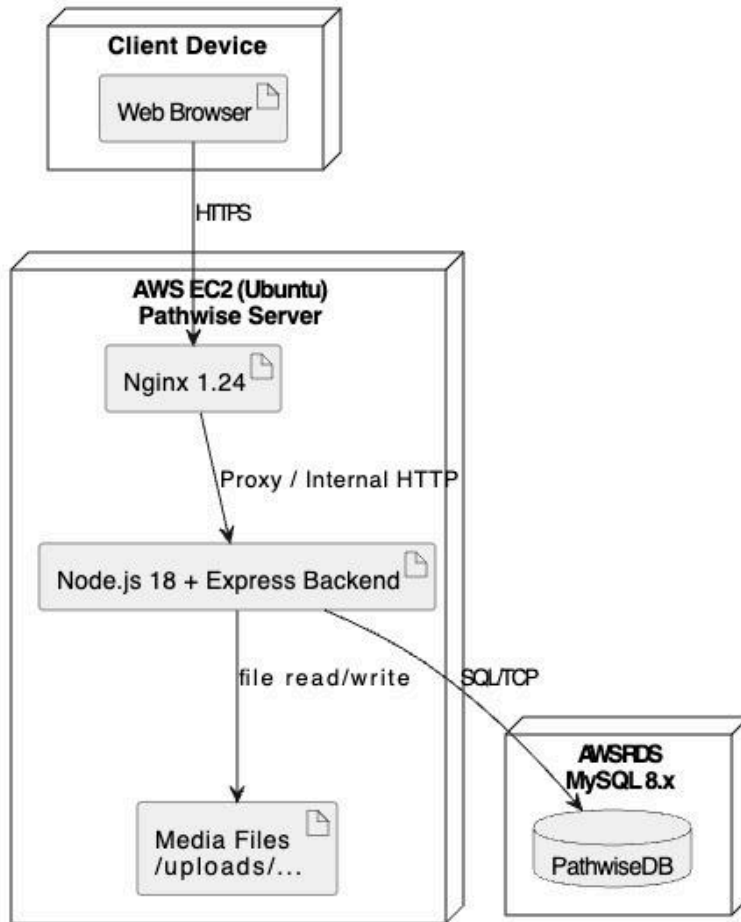
High Level UML Diagrams



Component Diagram



Deployment Diagram



Key Risks

- **Skill risks (do we have the right and enough skills):**
 - RISKS:
 - Since we are using full-stack dev tools like [Express.js](#), [Node.js](#), MySQL, Nginx configuration, and AWS deployment, there could be some team members who may not have any experience in some or most of them. So, some team members could potentially need some help to get up to speed.
 - Since we need advanced features such as dashboard analytics, activity aggregation, filtering search capability and ranking based on relevance, we need skills in backend logic and how to optimize our databases (tables). We need to implement these in the business logic layer to make sure we get accurate results, and correct data processing.
 - Then there is the part where we need to implement secure authentication and ensure privacy, which both require understanding of security practices. So this adds another layer of a different domain. So we need to make sure we don't over engineer the privacy aspect of our system.
 - UI/UX design should be simple enough that the backend could implement the logic and feel intuitive (think like boxes). So it's usable across browsers.
 - MITIGATION:
 - We could assign roles based on where our strengths lie (from backend, frontend, database, DevOps, or just a really good writer that can do a major part of the documentation).
 - We could share knowledge of what we know to others that don't (like how to navigate AWS, how to implement authentication, and deployment).
 - We should mainly use the official documentation for the technology we are using and the class resource as references when problems occur.
 - We should ensure Priority 1 requirement is first done before advancing to the other complicated features later on.
 - Instead of the stem parts being done separately (frontend, backend, databases, and deployment) and connecting these parts at the end, we should integrate small changes we make, so we can identify problems early on. This helps us to see what works together, if anyone lacks knowledge on a critical area or some unknown technical problem occurs.
- **Schedule risks (can we finish on time given what we committed to and the resources we are using)**
 - RISKS:
 - This project is very extensive with custom code (Goals, Projects, Milestones, Resources, Templates, Workflows, Messaging, Reflections, Search, Dashboard, and finally Compliance view) and already existing

products being integrated with it (AI, Google Maps, Google analytics). So this is a lot.

- Priority 2 and Priority 3 require much more time and could stretch the development time.
- Most importantly, members are taking other very advanced classes, possibly jobs, and personal matters. So we need to account for all these factors to offload work among ourselves intelligently and fairly.
- Debugging complex systems is a known time consuming part of development because of the speed required to build this product. Integration and testing would also take a long time to fit different components in the desired way.
- Also, there could be deployment configuration problems that can arise with Pathwise.

○ MITIGATION:

- Strictly and no exceptions, we should only focus on Priority 1 (the must have) features first.
- Also we should avoid adding unplanned features, and limit our scopes early.
- We should make it important to follow the rules of incremental development, so we find problems faster, and fix them faster. So in this way, we are overwhelmed by this one class alone.
- We should do early deployment to see if the platform is fit for the product.
- Since integrating AI might require some more additional knowledge we should keep it very modular and not write so much code to fit it into our product. It should be one where its very simple to work with any environment and has its own adapters (that is to say the devs did a lot of work to make it work on many codebases and integrates well).

- The only risk that is possible is unnecessary features that might delay finishing our product like AI and recommendation features.

● **Technical risks (any issues that occurred with the tools being used)**

○ RISKS:

- MySQL with LIKE operator for pattern matching may not scale as efficiently or provide strong reliability for fast ranking as the database grows. The performance limitation occurs when leading wildcards (LIKE '%term%') which would require full table scans. Also there is no ranking mechanism that is contextual with LIKE, as it only does literal pattern matching.
- There is also the risk of security vulnerabilities with session-based authentication if it's not properly implemented.

- There is a possibility with networking configuration issues that can occur with AWS EC2 + RDS instances. These are like security groups that are the main network firewalls for both of the instances, ports misconfiguration, Network ACLs permissions (even if security groups allow it).
 - Also, images stored in the server can cause path or permission issues, if image file names are changed or the storage structure is modified.
 - Browser compatibility can also occur if the frameworks used aren't supported across the majors like Chrome, Firefox, and Safari.
 - Since this is a full stack project, there is the case for unexpected bugs across frontend, backend and database layers.
- MITIGATION:
 - Keep our search and filtering simple for the vertical prototype
 - Also implemented baseline security approaches like secure sessions, input validation and hashed passwords.
 - Use A/B and staging/testing to see how the project is handled across platforms before the actual deploying the product.
 - Since this is a team led project, have error handling in place, so it can quicker to fix bugs. Logging will also help to analyze the error dump correctly.
 - Early on, we should test if our code works across the major browsers, so later on we don't waste time reverting major parts of our code.
 - Since there is the possibility for our databases being corrupted during testing, we should backup regularly.
- **Teamwork risks (is there a potential for any teamwork issue to arise)**
 - RISKS:
 - Maybe some members are getting an unproportional workload or their specific role requires more members' contribution.
 - There can also be a miscommunication about the terminology used in our approach to the project (like Goals vs Objects, Templates vs Blueprints, etc.).
 - There are also risks for severe merge conflict (big one) if not communicating on what work we are partitioning among ourselves.
 - There can also be the case, if one member component or role is not getting done, could be a deadlock for others since they are waiting on that part being done.
 - Conflicts can also arise with the design decision, maybe the UI team wants it in a particular way but it's hard or time-consuming for the backend to implement.
 - MITIGATION:

- We have already done this, but clearly assigning everyone their responsibilities (like frontend lead, backend lead and database manager) is very essential for basic start on this project. Then afterward we can self-organize and modify roles based on skill, schedule and technical complexity.
 - We could follow the standardized terminology that is laid out in the documentation.
 - To lessen and work in parallel, we can use the already standard Github branching strategy like feature branching and merging with pull requests to the main branch.
 - We can constantly talk on what we individually are working on, and having specific discord channels for certain parts of the project.
- **Legal/content risks (are the tools, or any content being used be obtained without any legal ramifications or do we need proper licenses)**
 - RISKS:
 - First there is the case we use third-party resources (like templates) intentionally or unintentionally without proper attributes.
 - Since AI is used to personalize the product for the users, there could be points the usage is high or even worse we need API licensing restriction.
 - Since we are using both Google Maps and Google Analytics, they require compliance with their terms of service. So we need to make sure we aren't breaking any of their terms.
 - Storing user data requires us to be careful but also ensure privacy protection and clear terms of services and clear communication of the policy.
 - Any of the images or content used should not flag any copyright, since there could be legal ramification tied with it.
 - MITIGATION:
 - We should use open-source libraries and resources that have very open licences.
 - Follow AWS, Google Maps, and Google Analytics terms of services.
 - We should have our own clear policy document that lays out clearly how we handle user data.
 - We should avoid using copyrighted material without permissions to risk not getting caught with copyright risks.
 - We should not implement payment services
 - We should mark our projects as a demonstration for a class and not a full commercial product.

Project Management

The Pathwise team managed M2 tasks using [Monday.com](https://monday.com), organized into boards corresponding to each document section: Functional Requirements, UI Mockups and Storyboards, Architecture and Database, UML Diagrams, Risks, and Vertical Prototype. Each task was assigned to a specific team member with a status tracked as Not Started, Working on It, or Done. This gave the team a unified view of progress and made it easy to identify blockers early.

To improve parallel efficiency, front-end and back-end responsibilities were separated early in M2. The front-end team (Damian, Cielina, Brandon) owned all UI mockups and storyboards independently, while the back-end team (Jason, Omeid) drove the vertical prototype and database architecture. A CSS and design standards task was completed by the front-end lead to establish shared component conventions before any implementation began. Front-end and back-end teams agreed on API interfaces as a coordination point rather than waiting for one side to finish before the other could start.

For M3 and beyond, the team will continue using Monday.com with the same structure, adding due dates to each task as milestone deadlines are confirmed. Priority 1 requirements will be tracked to completion before any Priority 2 work begins.

Team and Roles

Role	Team Member
Team Lead & Database Engineer	Omeid Nadery
GitHub Master	Laura Wong
Front End Lead	Damian Perez
Front End Support	Cielina Lubrino
Front End Support	Brandon Sanchez
Back End Lead	Jason Le
Backend Support	Mikias Berhane

Appendix I

Item	Credentials
Website URL	http://3.135.184.72/
Website URL to search page	http://3.135.184.72/verticalTestPage.html
SSH URL (from repo root)	ssh -i credentials/csc648-team-key.pem ubuntu@3.135.184.72
SSH Username	team-admin
SSH Password/Key	Pass: sfsu648! Key file: csc648-team-key.pem
Database URL	pathwise-db.c18i0cua0e4f.us-east-2.rds.amazonaws.com
Database Username	team2
Database Password	sfsu648!
Link to Github Page that Performs the Search	https://github.com/CSC-648-SFSU/CSC-648-848-S02-Spring2026-Team02/blob/bf68def069627f19a3f4b64d5e52cfc7f3d4e356/application/vertical-prototype/index.html

.env Information (For Reference):

DB_HOST=pathwise-db.c18i0cua0e4f.us-east-2.rds.amazonaws.com
DB_PORT=3306
DB_USER=team2
DB_PASSWORD=sfsu648!
DB_NAME=pathwise

Additional Info: The search flow is working end-to-end. The vertical prototype search page and backend test page both call the live API. Nginx proxies /api/ to Node on EC2, and Node queries the pathwise database on RDS. You can test keyword search, category filter, or both, and see results as cards (vertical prototype) or raw JSON (test page).

Search page: <http://3.135.184.72/vertical-prototype/index.html>

Backend test page (raw JSON): <http://3.135.184.72/verticalTestPage.html>

API health check: <http://3.135.184.72/api/db-test>

Search terms that return results: resume, interview, internship, study, office hours, guide, checklist, template, prep. Use the category dropdown (Academic, Career, Personal, Wellness) alone or with a keyword. For no results, try: xyzzz or nonexistent.